Instituto Politécnico Nacional

Escuela Superior de Cómputo

Unidad Académica:

Sistemas Distribuidos

Profesor:

Carreto Arellano Chadwick

Práctica 02: "Escalando al modelo Cliente-servidor"

Alumna:

Maldonado Bedolla Daniela del Carmen

Grupo:

7CM1

23 de febrero de 2026

Índice

INTRODUCCIÓN **3**

DESARROLLO **4**

PROBLEMÁTICA 4
PROPUESTA DE SOLUCIÓN 4
IMPLEMENTACIÓN 5
Generalidades expuestas en la práctica 1 ..... *5*
Servidor *5*
Cliente (partes críticas) *7*

EJECUCIÓN Y RESULTADOS **10**

EJECUCIÓN DE LA SIMULACIÓN CLIENTE/SERVIDOR 10
COMPARACIÓN DE RESULTADOS: PRÁCTICA 1 Y PRÁCTICA 2 13

CONCLUSIONES **14**

ANEXO **15**

CÓDIGO DEL SERVIDOR 15
CÓDIGO DE CLIENTE 16

INTRODUCCIÓN

La evolución de sistemas concurrentes hacia arquitecturas distribuidas representa un paso fundamental en el diseño de aplicaciones modernas. En la práctica 1, el problema de compra de boletos se resolvió en un entorno local mediante hilos dentro de un mismo proceso, donde múltiples compradores competían por un conjunto finito de asientos compartiendo memoria. Este enfoque permitió analizar condiciones de carrera, sincronización y consistencia de datos en un contexto controlado.

En la práctica 2, este mismo problema se amplía al modelo cliente/servidor, separando responsabilidades y trasladando la administración del recurso crítico al servidor. Bajo esta arquitectura, el servidor se convierte en la única autoridad sobre el estado de los asientos, mientras que el cliente genera hilos compradores que se comunican por sockets para solicitar, apartar y confirmar compras. Esta separación introduce un entorno más realista de concurrencia distribuida, donde la coordinación ya no depende de memoria compartida directa, sino de intercambio de mensajes y validaciones centralizadas.

A diferencia del modelo local, el esquema cliente/servidor requiere protocolos de comunicación, control de estado remoto y reglas claras de sincronización para evitar compras dobles o inconsistencias. Para ello, se implementan mecanismos de bloqueo en el servidor, reservas temporales, validación de propiedad de reserva y confirmación de compra, garantizando que cada asiento pase por estados consistentes durante toda la ejecución.

Asimismo, la práctica incorpora observabilidad del sistema mediante visualización en tiempo real y métricas de desempeño, incluyendo tiempos de búsqueda, compra y ejecución general. Esto permite comparar el comportamiento entre la solución local y la distribuida, identificando cómo impacta la comunicación en red sobre la concurrencia, y evidenciando la importancia de un control centralizado para mantener integridad y trazabilidad en escenarios de alta competencia por recursos finitos.

Desarrollo

Problemática

En la práctica 2 el problema ya no ocurre en un solo programa, sino en un esquema cliente/servidor.
Se tienen 1,000 asientos (25 x 40) y 2,000 compradores. Los compradores se crean como hilos en el cliente, pero los asientos los controla solo el servidor. Cada hilo intenta conseguir un boleto y confirmar su compra.

Aquí la dificultad principal es coordinar concurrencia **por red** sin perder consistencia. Los riesgos más importantes son:

- Dos clientes pueden intentar apartar o comprar el mismo asiento casi al mismo tiempo.
- Puede haber desfase entre lo que cree el cliente y el estado real del servidor.
- Si un cliente aparta y no confirma, ese asiento puede quedar "atorado" si no se libera.
- En la etapa final (cuando quedan pocos asientos) aumenta mucho la competencia y los intentos fallidos.

Por eso el reto no es solo la concurrencia de hilos, sino también mantener un estado único y correcto en el servidor durante toda la venta.

Propuesta de solución

La solución implementada usa un modelo cliente/servidor donde el servidor es la autoridad total del sistema:

- El cliente crea los 2,000 hilos compradores y cada hilo manda solicitudes por socket.
- El servidor valida todo: solicitud de asiento, apartado y compra final.
- Los asientos se manejan con estados (FREE, RESERVED, SOLD) y control de exclusión para evitar doble venta.
- El apartado tiene tiempo límite (TTL); si no se confirma a tiempo, el asiento se libera automáticamente.

- La compra solo se permite al mismo comprador que hizo la reserva (validación de dueño de reserva).
- Al llegar a 1,000 vendidos, el servidor marca venta terminada y deja de aceptar nuevas compras.

Además, se agregó visualización en el servidor y métricas de tiempo para observar el comportamiento del sistema.
Con este enfoque se logra que no haya ventas duplicadas, que el conteo final sea correcto y que el cierre de ejecución sea limpio tanto en servidor como en cliente.

Implementación

Generalidades expuestas en la práctica 1

El programa modela un foro como una matriz de **25 filas x 40 columnas**, para un total de **1,000 asientos**. Cada posición de la matriz representa un asiento único y su estado se guarda como un valor booleano: disponible o vendido. Esta estructura permite ubicar y actualizar asientos de forma directa por coordenadas (fila, columna).

```
FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
COMPRADORES_POR_ASIENTO = 2
TOTAL_COMPRADORES = TOTAL_ASIENTOS * COMPRADORES_POR_ASIENTO
```

Servidor

El servidor hace función de el organizador con control total de la venta de boletos, a continuación, se resaltan las partes críticas del código.

1. **Inicio de venta controlado por el servidor**

```
def open_sales(self):
    with self.state_lock:
        if self.sales_open_event.is_set():
            return
        self.sales_started_at = time.perf_counter()
        self.sales_open_event.set()

    with self.terminal_lock:
        print("Actualizaciones de venta:")
```

- *sales_open_event: evita que la venta inicie dos veces.*
- *sales_started_at: marca inicio real para métricas.*
- *El servidor imprime y controla el arranque (no el cliente).*

2. Apartado de asiento (request_ticket)

```python
def request_ticket(self, buyer_id, request_id):
    with self.state_lock:
        if not self.sales_open_event.is_set():
            return {"status": "not_started", "message": "La venta aún no inicia."}

        self._cleanup_expired_locked()

        if self.sold_count >= TOTAL_ASIENTOS or not self.free_seats:
            return {"status": "sold_out", "message": "No hay asientos disponibles."}

        seat = random.choice(tuple(self.free_seats))
        row, col = seat
        reservation_id = str(uuid.uuid4())
        expires_at = time.monotonic() + RESERVA_TTL_SEGUNDOS

        self.free_seats.remove(seat)
        self.seat_status[row][col] = "RESERVED"
        self.reservations[reservation_id] = {
            "buyer_id": buyer_id,
            "seat": seat,
            "expires_at": expires_at,
        }

        return {
            "status": "ok",
            "reservation_id": reservation_id,
            "seat": {"row": row, "col": col},
            "ttl_seconds": RESERVA_TTL_SEGUNDOS,
        }
```

- *Si la venta no inicia: not_started.*
- *Si ya no hay lugares: sold_out.*
- *Reserva temporal con reservation_id + TTL.*

3. Compra final (purchase) con validaciones

```python
def purchase(self, buyer_id, request_id, reservation_id):
    with self.state_ (function) sales_open_event: Any
        if not self.sales_open_event.is_set():
            return {"status": "not_started", "message": "La venta aún no inicia."}

        self._cleanup_expired_locked()
        info = self.reservations.get(reservation_id)

        if info is None:
            return {"status": "error", "code": "invalid_or_expired_reservation"}

        if info["buyer_id"] != buyer_id:
            return {"status": "error", "code": "reservation_owner_mismatch"}

        row, col = info["seat"]
        self.reservations.pop(reservation_id, None)
        self.seat_status[row][col] = "SOLD"
        self.sold_count += 1
        self._report_progress_milestones_locked()

        if self.sold_count >= TOTAL_ASIENTOS and not self.sold_out_event.is_set():
            self.sales_finished_at = time.perf_counter()
            self.sold_out_event.set()

        return {
            "status": "ok",
            "seat": {"row": row, "col": col},
            "sold_count": self.sold_count,
            "remaining": TOTAL_ASIENTOS - self.sold_count,
        }
```

- *Evita compra doble y compra de reservas ajenas.*
- *Cambia estado RESERVED -> SOLD.*
- *Activa fin global cuando llega a 100%.*

4. **Liberación automática de reservas vencidas**

```python
def _cleanup_expired_locked(self):
    now = time.monotonic()
    expired = [
        rid for rid, info in self.reservations.items()
        if info["expires_at"] <= now
    ]

    for rid in expired:
        info = self.reservations.pop(rid)
        row, col = info["seat"]
        self.seat_status[row][col] = "FREE"
        self.free_seats.add((row, col))
```

- *Si no compran a tiempo, el asiento vuelve a FREE.*
- *Evita asientos "atorados".*

5. **Protocolo de sockets por acción**

```python
class TicketRequestHandler(socketserver.StreamRequestHandler):
    def handle(self):
        self.server.register_connection()
        while True:
            raw_line = self.rfile.readline()
            if not raw_line:
                return

            payload = json.loads(raw_line.decode("utf-8").strip())
            action = payload.get("action")
```

- *action define el flujo (request_ticket, purchase, health).*
- *Respuesta JSON por cada petición.*
- *health lo usa el cliente para esperar inicio de venta.*

Cliente (partes críticas)

1. **Envío de petición al servidor**

```python
def send_request(host, port, payload):
    data = (json.dumps(payload) + "\n").encode("utf-8")
    with socket.create_connection((host, port), timeout=SOCKET_TIMEOUT) as sock:
        sock.settimeout(SOCKET_TIMEOUT)
        sock.sendall(data)
        response_bytes = b""
        while not response_bytes.endswith(b"\n"):
            chunk = sock.recv(4096)
            if not chunk:
                break
            response_bytes += chunk

    if not response_bytes:
        raise ConnectionError("Respuesta vacía del servidor")

    return json.loads(response_bytes.decode("utf-8").strip())
```

- *Protocolo simple por línea JSON.*
- *Cada hilo cliente usa esta función para hablar con el servidor.*

2. Esperar a que el servidor abra la venta

```python
def wait_for_sales_start(host, port):
    print("Esperando señal de inicio del servidor...")
    while True:
        try:
            response = send_request(host, port, {"action": "health", "request_id": str(uuid.uuid4())})
        except Exception:
            time.sleep(0.4)
            continue

        if response.get("status") == "ok" and response.get("sales_open"):
            print("Servidor confirmó inicio de venta.")
            return

        time.sleep(0.3)
```

- *El cliente no decide cuándo iniciar.*
- *Espera sales_open=True desde servidor.*

3. Lógica de comprador-hilo

```python
def buyer_worker(buyer_id, host, port):
    while not sold_out_event.is_set():
        time.sleep(random.uniform(0.1, 2.0))

        request_payload = {
            "action": "request_ticket",
            "buyer_id": buyer_id,
            "request_id": str(uuid.uuid4()),
        }
        request_response = send_request(host, port, request_payload)
        request_status = request_response.get("status")

        if request_status in ("not_started",):
            time.sleep(0.2)
            continue
        if request_status == "sold_out":
            sold_out_event.set()
            break
        if request_status != "ok":
            continue

        purchase_payload = {
            "action": "purchase",
            "buyer_id": buyer_id,
            "reservation_id": request_response.get("reservation_id"),
            "request_id": str(uuid.uuid4()),
        }
```

- *Flujo real: buscar → apartar → comprar.*
- *Maneja respuestas clave: not_started, sold_out, ok.*

4. Creación de hilos compradores

```python
for buyer_id in range(1, TOTAL_COMPRADORES + 1):
    thread = threading.Thread(
        target=buyer_worker,
        args=(buyer_id, args.host, args.port),
        daemon=False
    )
    threads.append(thread)
    thread.start()
    time.sleep(0.0005)
```

- *TOTAL_COMPRADORES = TOTAL_ASIENTOS * 2.*
- *Cada hilo representa un comprador independiente.*

5. Resumen de métricas en cliente

```
def print_summary():
    print("========== Resumen de Tiempos del Cliente ==========")
    print(f"Compradores creados: {TOTAL_COMPRADORES}")
    print(f"Compradores con compra: {metrics['buyers_success']}")
    print(f"Compradores sin compra: {metrics['buyers_fail']}")
    print(f"Tiempo total general: {sales_end_ts - sales_start_ts:.4f} s")
    print(f"Tiempo promedio request_ticket: {avg(metrics['request_time_total'], metrics['request_count']):.
    print(f"Tiempo promedio purchase: {avg(metrics['purchase_time_total'], metrics['purchase_count']):.6f} s
```

- *Te da evidencia de rendimiento del lado cliente.*
- *Útil para análisis del reporte (latencias, éxito/fallo).*

Ejecución y Resultados

Ejecución de la simulación cliente/servidor

En esta práctica, la ejecución inicia con el **servidor**, que prepara el estado del foro (25x40), habilita la visualización y queda en espera de conexiones entrantes. En terminal se muestra el arranque del servicio, el puerto de escucha y la disponibilidad inicial de boletos. En esta etapa todavía no hay venta activa: el servidor permanece en espera hasta detectar al menos un cliente conectado.

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor> python .\servidor.py
Servidor de boletos iniciado
Escuchando en 127.0.0.1:5000
Asientos disponibles: 1000
Visualización: activada
```

Figura 1. Arranque del servidor en terminal

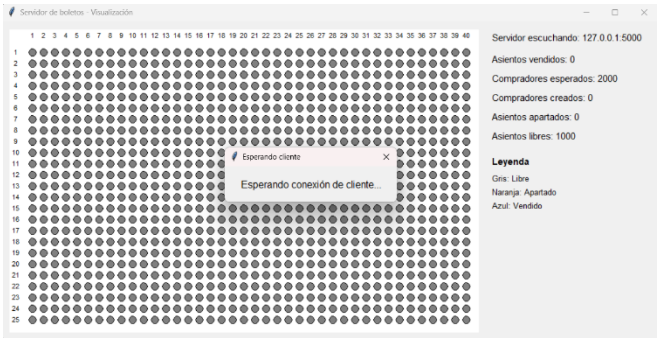

Figura 2. Visualización de la venta con el servidor en espera

Después, se ejecuta el **cliente**, que establece conexión por socket y espera la señal de inicio enviada por el servidor.
Al conectarse correctamente, el servidor cambia de estado a **"cliente conectado"**, muestra el mensaje de depuración y activa la cuenta regresiva de inicio de venta. Con esto se confirma que la comunicación cliente/servidor es correcta y que el sistema está listo para comenzar la compra concurrente.

```
imo\SistemasDistribuidos\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor> python .\cliente.py
Cliente de compra iniciado
Servidor destino: 127.0.0.1:5000
Asientos esperados: 1000
Compradores a crear: 2000
Esperando señal de inicio del servidor...
```

Figura 3. Ejecución del cliente

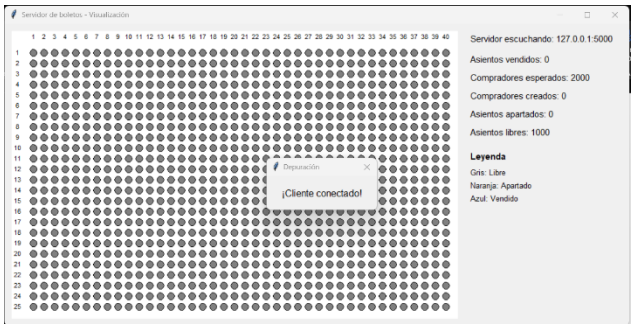

Figura 4. Cliente conectado e inicio de la cuenta regresiva

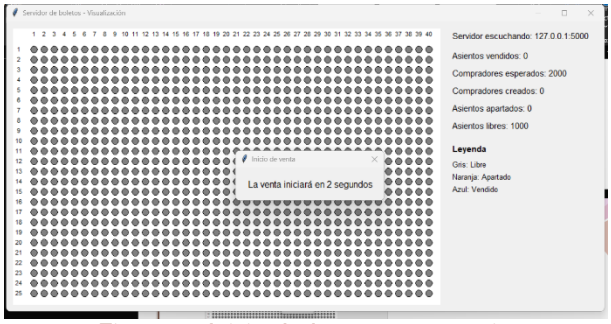

Figura 5. Inicio de la cuenta regresiva

Durante la ejecución, la simulación muestra de forma continua el cambio de estado de los asientos controlado por el servidor. Cada asiento inicia en gris (libre), pasa a naranja cuando se aparta temporalmente y cambia a azul cuando la compra se confirma. Esta secuencia permite observar claramente el flujo real de la transacción (disponible → reservado → vendido) bajo concurrencia.

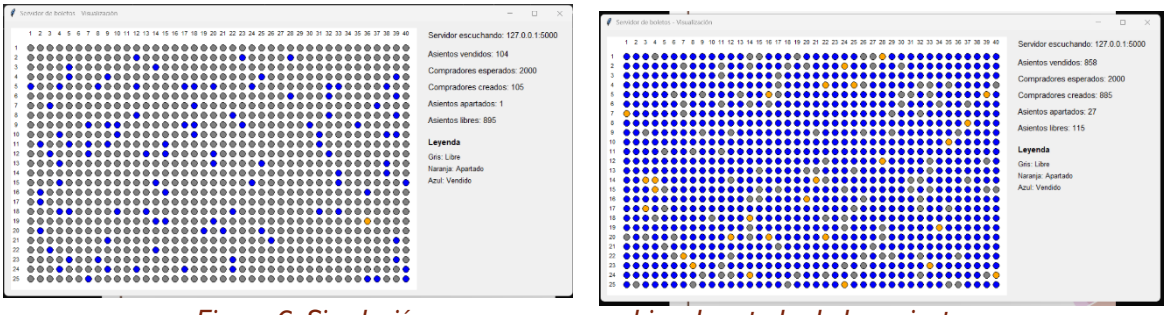

Figura 6. Simulación en curso con cambios de estado de los asientos

Mientras la venta avanza, el servidor mantiene la consistencia del estado y el cliente continúa procesando hilos compradores hasta agotar los boletos. En terminal se reportan también hitos de progreso (25%, 50%, 75% y 100%), pero en esta etapa se usan como apoyo de monitoreo, no como elemento principal de análisis.

Al llegar al 100% de asientos vendidos, el sistema marca la venta como finalizada, se muestra el mensaje **"La venta ha concluido"** y se realiza el cierre ordenado de la ejecución. Finalmente, se imprimen los resúmenes de procesamiento en ambos lados:

- **Servidor:** total vendido, compradores detectados, solicitudes atendidas y tiempos promedio de operaciones.
- **Cliente:** compradores con/sin compra, tiempos generales y métricas de request/purchase.

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor> python .\servidor.py
Servidor de boletos iniciado
Escuchando en 127.0.0.1:5000
Asientos disponibles: 1000
Visualización: activada
Actualizaciones de venta:
Actualización: 25% de boletos vendidos (250/1000).
Actualización: 50% de boletos vendidos (500/1000).
Actualización: 75% de boletos vendidos (750/1000).
Actualización: 100% de boletos vendidos (1000/1000).
Actualización: la venta ha concluido.
```

Figura 7. Actualizaciónes y finalización de la simulación

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor> python .\servidor.py
Asientos disponibles: 1000
Visualización: activada
Actualizaciones de venta:
Actualización: 25% de boletos vendidos (250/1000).
Actualización: 50% de boletos vendidos (500/1000).
Actualización: 75% de boletos vendidos (750/1000).
Actualización: 100% de boletos vendidos (1000/1000).
Actualización: la venta ha concluido.

========== Resumen del Servidor ==========
Asientos totales: 1000
Asientos vendidos: 1000
Compradores esperados (doble): 2000
Compradores creados (únicos): 1530
Asientos restantes: 0
Reservas activas: 0
Reservas expiradas liberadas: 0
Solicitudes antes del inicio: 0
Tiempo total de ejecución de venta: 2.9747 s
Request_ticket procesados: 1530
Compras procesadas: 1000
Request_ticket exitosos: 1000
Compras exitosas: 1000
Compras rechazadas: 0
Tiempo promedio request_ticket: 0.000035 s
Tiempo promedio purchase: 0.000015 s
==========

(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor>
```

Figura 8. Resumen del servidor

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor> python .\cliente.py
Cliente de compra iniciado
Servidor destino: 127.0.0.1:5000
Asientos esperados: 1000
Compradores a crear: 2000
Esperando señal de inicio del servidor...
Servidor confirmó inicio de venta.

========== Resumen de Tiempos del Cliente ==========
Compradores creados: 2000
Compradores con compra: 1000
Compradores sin compra: 1000
Tiempo total general: 4.1139 s
Promedio total por comprador exitoso: 0.8046 s
Promedio de búsqueda por comprador exitoso: 0.7816 s
Promedio de compra por comprador exitoso: 0.022607 s
Promedio de espera aleatoria por comprador exitoso: 0.7741 s
Promedio total por comprador sin compra: 1.3514 s
Promedio de espera aleatoria por comprador sin compra: 1.3366 s
Promedio de intentos por comprador exitoso: 1.00
Promedio de intentos por comprador sin compra: 1.00
Tiempo promedio request_ticket (red + servidor): 0.010953 s
Tiempo promedio purchase (red + servidor): 0.022607 s
Errores de red detectados: 0
==========

(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor>
```

Figura 9. Resumen del cliente

Comparación de resultados: práctica 1 y práctica 2

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\01-multiprocesamiento> python .\practica-01.py
Bienvenido a la compra de boletos del Foro Dani
Número de asientos disponibles: 1000
Número de compradores registrados: 2000
Actualizaciones de venta:
Actualización: 25% de boletos vendidos (250/1000).
Actualización: 50% de boletos vendidos (500/1000).
Actualización: 75% de boletos vendidos (750/1000).
Actualización: 100% de boletos vendidos (1000/1000).
Actualización: la venta ha concluido.
Resumen de tiempos:
- Tiempo total general de la simulación: 13.3488 s
- Promedio por comprador exitoso (buscar + comprar): 1.0912 s
- Promedio de búsqueda por comprador exitoso: 1.0912 s
- Promedio de compra por comprador exitoso: 0.000002 s
- Promedio de espera aleatoria por comprador exitoso: 1.0908 s
- Promedio de intentos por comprador exitoso: 1.30
- Promedio por comprador sin compra: 10.7453 s
- Promedio de espera aleatoria por comprador sin compra: 10.7422 s
- Promedio de intentos por comprador sin compra: 9.92
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\01-multiprocesamiento>
```

Figura 10. Medicion de tiempo en práctica 1.

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor> python .\servidor.py
Asientos disponibles: 1000
Visualización: activada
Actualizaciones de venta:
Actualización: 25% de boletos vendidos (250/1000).
Actualización: 50% de boletos vendidos (500/1000).
Actualización: 75% de boletos vendidos (750/1000).
Actualización: 100% de boletos vendidos (1000/1000).
Actualización: la venta ha concluido.

========== Resumen del Servidor ==========
Asientos totales: 1000
Asientos vendidos: 1000
Compradores esperados (doble): 2000
Compradores creados (únicos): 1530
Asientos restantes: 0
Reservas activas: 0
Reservas expiradas liberadas: 0
Solicitudes antes del inicio: 0
Tiempo total de ejecución de venta: 2.9747 s
Request_ticket procesados: 1530
Compras procesadas: 1000
Request_ticket exitosos: 1000
Compras exitosas: 1000
Compras rechazadas: 0
Tiempo promedio request_ticket: 0.000035 s
Tiempo promedio purchase: 0.000015 s
==========
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\02-clienteservidor>
```

Figura 11. Medicion de tiempo en práctica 2.

En ambas prácticas se logró el objetivo principal: vender los 1,000 boletos sin sobreventa y con cierre correcto del sistema. También en ambos casos los hitos de avance (25%, 50%, 75% y 100%) se cumplieron en orden, lo que confirma consistencia en el conteo global de asientos vendidos.

En la **práctica 1** (todo en un solo programa), el tiempo total fue de **13.3488 s**. El comportamiento muestra una presión alta al final: los compradores sin compra tardan más en promedio (**10.7453 s**) y realizan más intentos (**9.92**), porque siguen compitiendo localmente hasta que se agotan los asientos.

En la **práctica 2** (cliente-servidor), el servidor reportó un tiempo de venta de **2.9747 s**, con **1000 compras exitosas** y **0 compras rechazadas**. Además, no quedaron reservas activas ni liberaciones pendientes, lo que indica que el flujo de apartado/compra terminó limpio.

Un cambio importante es que en práctica 2 el servidor registró **1530 compradores únicos** (de 2000 esperados), porque no todos alcanzan a solicitar asiento antes de que la venta termine; esto es normal en este modelo cuando el cierre ocurre rápido y el servidor ya marcó sold_out.

Como conclusión, la práctica 1 sirve mejor para observar la competencia intensa entre hilos en memoria compartida, mientras que la práctica 2 demuestra mejor control transaccional centralizado y cierre ordenado en un entorno distribuido. Aunque los tiempos no son 100% equivalentes (porque se miden distinto en cada práctica), los resultados validan que el modelo cliente-servidor mantiene integridad y mejora la gestión del proceso de venta.

Conclusiones

Después de completar esta práctica, entiendo mucho mejor la diferencia entre resolver concurrencia en un programa local y hacerlo en un modelo cliente-servidor. En la práctica 1 aprendí a controlar condiciones de carrera con hilos compartiendo memoria; en la práctica 2 confirmé que, al distribuir el sistema, la clave es definir una autoridad clara del estado, y en este caso esa autoridad fue el servidor.

También me quedó más claro que la concurrencia no se resuelve solo con "más hilos", sino con reglas de control bien definidas: quién puede apartar, quién puede comprar, cuándo expira una reserva y cómo cerrar el sistema sin inconsistencias. Implementar estos pasos evitó sobreventa, mantuvo el conteo correcto y permitió finalizar la ejecución de forma ordenada en cliente y servidor.

Otro punto importante fue la observabilidad. Poder ver los cambios de estado en la simulación (libre, apartado, vendido), junto con los hitos de venta y los resúmenes de tiempos, me ayudó a entender mejor qué estaba pasando en cada etapa del proceso y a validar que el comportamiento era consistente bajo alta competencia.

Aunque el problema sigue siendo académico, esta versión ya se acerca más a un escenario real. Queda claro que para escalar en futuras prácticas se necesitarán más componentes (por ejemplo, pagos, múltiples boletos por cliente, comprobantes y posiblemente varios servidores), pero esta base ya me dejó una idea sólida de cómo separar responsabilidades y proteger recursos críticos.

En conclusión, esta práctica me ayudó a pasar de una visión "local" de concurrencia a una visión distribuida más completa: no basta con ejecutar tareas en paralelo, hay que coordinar estados, comunicar procesos correctamente y mantener integridad de datos de principio a fin.

Anexo

Nota: Debido a la extensión de ambos códigos, tanto cliente.py como servidor.py se anexarán a la carpeta de entrega, en este reporte solo se incluyeron capturas de secciones iniciales del código.

Código del servidor

```
import argparse
import json
import random
import socketserver
import threading
import time
import tkinter as tk
import uuid

FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
TOTAL_COMPRADORES_ESPERADOS = TOTAL_ASIENTOS * 2
RESERVA_TTL_SECONDS = 3.0


class TicketState:
    def __init__(self):
        self.state_lock = threading.Lock()
        self.terminal_lock = threading.Lock()
        self.seat_status = ["FREE" for _ in range(COLUMNAS)] for _ in range(FILAS)]
        self.free_seats = [(r, c) for r in range(FILAS) for c in range(COLUMNAS)]
        self.reservations = {}
        self.sold_count = 0

        self.sales_started_at = None
        self.sales_finished_at = None
        self.sold_out_event = threading.Event()
        self.sales_open_event = threading.Event()
        self.summary_printed = False

        self.history = []
        self.unique_buyers = set()
        self.hitos_reportados = set()

        self.metrics = {
            "request_ticket_count": 0,
            "purchase_count": 0,
            "request_ticket_time_total": 0.0,
            "purchase_time_total": 0.0,
            "request_ticket_ok": 0,
            "purchase_ok": 0,
            "purchase_rejected": 0,
            "expired_releases": 0,
            "not_started_count": 0,
        }

    def log_event(self, request_id, buyer_id, action, result, seat=None, reservation_id=None, extra=None):
        event = {
            "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
            "request_id": request_id,
            "buyer_id": buyer_id,
            "action": action,
            "result": result,
            "seat": seat,
            "reservation_id": reservation_id,
        }
        if extra is not None:
            event["extra"] = extra
        self.history.append(event)

    def _cleanup_expired_locked(self):
        now = time.monotonic()
        expired = [
            reservation_id
            for reservation_id, info in self.reservations.items()
            if info["expires_at"] <= now
        ]

        for reservation_id in expired:
            info = self.reservations.pop(reservation_id)
            row, col = info["seat"]
            self.seat_status[row][col] = "FREE"
            self.free_seats.add((row, col))
            self.metrics["expired_releases"] += 1
            self._log_event(
                request_id=f"server-expire-{reservation_id}",
                buyer_id=info["buyer_id"],
                action="EXPIRED_RESERVATION",
                result="RELEASED",
                seat={"row": row, "col": col},
                reservation_id=reservation_id,
            )

    def open_sales(self):
        with self.state_lock:
            if self.sales_open_event.is_set():
                return
            self.sales_started_at = time.perf_counter()
            self.sales_open_event.set()

        with self.terminal_lock:
            print("Actualizaciones de venta:")

    def _report_progress_milestones_locked(self):
        for porcentaje, umbral in (
            (25, int(TOTAL_ASIENTOS * 0.25)),
            (50, int(TOTAL_ASIENTOS * 0.50)),
            (75, int(TOTAL_ASIENTOS * 0.75)),
            (100, TOTAL_ASIENTOS),
        ):
            if self.sold_count >= umbral and porcentaje not in self.hitos_reportados:
                self.hitos_reportados.add(porcentaje)
                with self.terminal_lock:
                    print(f"Actualización: {porcentaje}% de boletos vendidos ({self.sold_count}/{TOTAL_ASIENTOS}).")

    def register_buyer(self, buyer_id):
        if buyer_id is None:
            return
        with self.state_lock:
            self.unique_buyers.add(str(buyer_id))

    def sales_open(self):
        return self.sales_open_event.is_set()
```

```
def request_ticket(self, buyer_id, request_id):
    started = time.perf_counter()

    with self.state_lock:
        if not self.sales_open_event.is_set():
            self.metrics["not_started_count"] += 1
            response = {
                "status": "not_started",
                "message": "La venta aún no inicia.",
            }
        else:
            self._cleanup_expired_locked()
            self.metrics["request_ticket_count"] += 1

            if self.sold_count >= TOTAL_ASIENTOS or not self.free_seats:
                self._log_event(request_id, buyer_id, "REQUEST_TICKET", "SOLD_OUT")
                response = {
                    "status": "sold_out",
                    "message": "No hay asientos disponibles.",
                }
            else:
                seat = random.choice(tuple(self.free_seats))
                row, col = seat
                reservation_id = str(uuid.uuid4())
                expires_at = time.monotonic() + RESERVA_TTL_SECONDS

                self.free_seats.remove(seat)
                self.seat_status[row][col] = "RESERVED"
                self.reservations[reservation_id] = {
                    "buyer_id": buyer_id,
                    "seat": seat,
                    "expires_at": expires_at,
                }

                self.metrics["request_ticket_ok"] += 1
                self._log_event(
                    request_id,
                    buyer_id,
                    "REQUEST_TICKET",
                    "RESERVED",
                    seat={"row": row, "col": col},
                    reservation_id=reservation_id,
                )
                response = {
                    "status": "ok",
                    "reservation_id": reservation_id,
                    "seat": {"row": row, "col": col},
                    "ttl_seconds": RESERVA_TTL_SECONDS,
                }

    elapsed = time.perf_counter() - started
    self.metrics["request_ticket_time_total"] += elapsed
    return response

def purchase(self, buyer_id, request_id, reservation_id):
    started = time.perf_counter()

    with self.state_lock:
        if not self.sales_open_event.is_set():
            self.metrics["not_started_count"] += 1
            response = {
                "status": "not_started",
                "message": "La venta aún no inicia.",
            }
        else:
            self._cleanup_expired_locked()
            self.metrics["purchase count"] += 1

            if self.sold_count >= TOTAL_ASIENTOS:
                self._log_event(request_id, buyer_id, "PURCHASE", "SOLD_OUT", reservation_id=reservation_id)
                response = {
                    "status": "sold_out",
                    "message": "La venta terminó.",
                }
            else:
                info = self.reservations.pop(reservation_id)
                if info is None:
                    self.metrics["purchase_rejected"] += 1
                    self._log_event(
                        request_id,
                        buyer_id,
                        "PURCHASE",
                        "EXPIRED_OR_INVALID_RESERVATION",
                        reservation_id=reservation_id,
                    )
                    response = {
                        "status": "error",
                        "code": "EXPIRED_OR_INVALID_RESERVATION",
                    }
                elif info["buyer_id"] != buyer_id:
                    self.metrics["purchase_rejected"] += 1
                    row, col = info["seat"]
                    self._log_event(
                        request_id,
                        buyer_id,
                        "PURCHASE",
                        "FORBIDDEN",
                        seat={"row": row, "col": col},
                        reservation_id=reservation_id,
                    )
                    response = {
                        "status": "error",
                        "code": "reservation_owner_mismatch",
                    }
                else:
                    row, col = info["seat"]
                    self.reservations.setdefault(reservation_id, None)
                    self.seat_status[row][col] = "SOLD"
                    self.sold_count += 1
                    self._report_progress_milestones_locked()
                    self.metrics["purchase_ok"] += 1
                    self._log_event(
                        request_id,
                        buyer_id,
                        "PURCHASE",
                        "OK",
                        seat={"row": row, "col": col},
                        reservation_id=reservation_id,
                    )

                    if self.sold_count >= TOTAL_ASIENTOS and not self.sold_out_event.is_set():
                        self.sales_finished_at = time.perf_counter()
                        self.sold_out_event.set()

                    response = {
                        "status": "ok",
                        "seat": {"row": row, "col": col},
                        "sold_count": self.sold_count,
                        "remaining": TOTAL_ASIENTOS - self.sold_count,
                    }

    elapsed = time.perf_counter() - started
    self.metrics["purchase_time_total"] += elapsed
    return response

def get_snapshot(self):
    with self.state_lock:
        reserved_count = len(self.reservations)
        free_count = TOTAL_ASIENTOS - self.sold_count - reserved_count
        seat_status_copy = [row[:] for row in self.seat_status]
        return {
            "sold_count": self.sold_count,
            "reserved_count": reserved_count,
            "free_count": free_count,
            "seat_status": seat_status_copy,
            "buyers_created": len(self.unique_buyers),
        }

def print_summary_once(self):
    with self.state_lock:
        if self.summary_printed:
            return
        self.summary_printed = True
    self._print_summary()
```

Código de cliente

```python
import argparse
import json
import random
import socket
import threading
import time
import uuid

FILAS = 26
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
COMPRADORES_POR_ASIENTO = 2
TOTAL_COMPRADORES = TOTAL_ASIENTOS * COMPRADORES_POR_ASIENTO

SOCKET_TIMEOUT = 2.0


stats_lock = threading.Lock()
terminal_lock = threading.Lock()
sold_out_event = threading.Event()
threads = []

metrics = {
    "buyers_success": 0,
    "buyers_fail": 0,
    "buyer_total_success": 0.0,
    "buyer_search_success": 0.0,
    "buyer_purchase_success": 0.0,
    "buyer_wait_success": 0.0,
    "buyer_total_fail": 0.0,
    "buyer_wait_fail": 0.0,
    "request_time_total": 0.0,
    "purchase_time_total": 0.0,
    "request_count": 0,
    "purchase_count": 0,
    "attempts_success": 0,
    "attempts_fail": 0,
    "network_errors": 0,
}

sales_start_ts = None
sales_end_ts = None


def avg(total, count):
    if count == 0:
        return 0.0
    return total / count


def send_request(host, port, payload):
    data = (json.dumps(payload) + "\n").encode("utf-8")
    with socket.create_connection((host, port), timeout=SOCKET_TIMEOUT) as sock:
        sock.settimeout(SOCKET_TIMEOUT)
        sock.sendall(data)
        response_bytes = b""
        while not response_bytes.endswith(b"\n"):
            chunk = sock.recv(4096)
            if not chunk:
                break
            response_bytes += chunk

    if not response_bytes:
        raise ConnectionError("Respuesta vacía del servidor")

    return json.loads(response_bytes.decode("utf-8").strip())


def buyer_worker(buyer_id, host, port):
    buyer_started = time.perf_counter()
    wait_random_accum = 0.0
    request_time_accum = 0.0
    purchase_time_accum = 0.0
    attempts = 0
    purchased = False

    while not sold_out_event.is_set():
        pause = random.uniform(0.1, 2.0)
        time.sleep(pause)
        wait_random_accum += pause

        request_payload = {
            "action": "request_ticket",
            "buyer_id": buyer_id,
            "request_id": str(uuid.uuid4()),
        }

        attempts += 1
        request_started = time.perf_counter()
        try:
            response = send_request(host, port, request_payload)
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            continue

        request_elapsed = time.perf_counter() - request_started
        request_time_accum += request_elapsed

        with stats_lock:
            metrics["request_count"] += 1
            metrics["request_time_total"] += request_elapsed

        request_status = response.get("status")
        if request_status == "not_started":
            time.sleep(0.2)
            continue
        if request_status == "sold_out":
            sold_out_event.set()
            break
        if request_status != "ok":
            continue

        reservation_id = response.get("reservation_id")
        if not reservation_id:
            continue

        purchase_payload = {
            "action": "purchase",
            "buyer_id": buyer_id,
            "reservation_id": reservation_id,
            "request_id": str(uuid.uuid4()),
        }

        purchase_started = time.perf_counter()
        try:
            purchase_response = send_request(host, port, purchase_payload)
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            continue

        purchase_elapsed = time.perf_counter() - purchase_started
        purchase_time_accum += purchase_elapsed

        with stats_lock:
            metrics["purchase_count"] += 1
            metrics["purchase_time_total"] += purchase_elapsed

        purchase_status = purchase_response.get("status")
        if purchase_status == "not_started":
            time.sleep(0.2)
            continue
        if purchase_status == "sold_out":
            sold_out_event.set()
            break
        if purchase_status == "ok":
            purchased = True
            sold_count = purchase_response.get("sold_count", 0)
            if sold_count >= TOTAL_ASIENTOS:
                sold_out_event.set()
            break

    buyer_total_elapsed = time.perf_counter() - buyer_started
    buyer_search_elapsed = wait_random_accum + request_time_accum

    with stats_lock:
        if purchased:
            metrics["buyers_success"] += 1
            metrics["buyer_total_success"] += buyer_total_elapsed
            metrics["buyer_search_success"] += buyer_search_elapsed
            metrics["buyer_purchase_success"] += purchase_time_accum
            metrics["buyer_wait_success"] += wait_random_accum
            metrics["attempts_success"] += attempts
        else:
            metrics["buyers_fail"] += 1
            metrics["buyer_total_fail"] += buyer_total_elapsed
            metrics["buyer_wait_fail"] += wait_random_accum
            metrics["attempts_fail"] += attempts
```

```python
def print_summary():
    success = metrics["buyers_success"]
    fail = metrics["buyers_fail"]

    with terminal_lock:
        print("========== Resumen de Tiempos del Cliente =========")
        print(f"Compradores creados: {TOTAL_COMPRADORES}")
        print(f"Compradores con compra: {success}")
        print(f"Compradores sin compra: {fail}")
        print(f"Tiempo total general: {sales_end_ts - sales_start_ts:.4f} s")
        print(
            f"Promedio total por comprador exitoso: "
            f"{avg(metrics['buyer_total_success'], success):.4f} s"
        )
        print(
            f"Promedio de búsqueda por comprador exitoso: "
            f"{avg(metrics['buyer_search_success'], success):.4f} s"
        )
        print(
            f"Promedio de compra por comprador exitoso: "
            f"{avg(metrics['buyer_purchase_success'], success):.6f} s"
        )
        print(
            f"Promedio de espera aleatoria por comprador exitoso: "
            f"{avg(metrics['buyer_wait_success'], success):.4f} s"
        )
        print(
            f"Promedio total por comprador sin compra: "
            f"{avg(metrics['buyer_total_fail'], fail):.4f} s"
        )
        print(
            f"Promedio de espera aleatoria por comprador sin compra: "
            f"{avg(metrics['buyer_wait_fail'], fail):.4f} s"
        )
        print(
            f"Promedio de intentos por comprador exitoso: "
            f"{avg(metrics['attempts_success'], success):.2f}"
        )
        print(
            f"Promedio de intentos por comprador sin compra: "
            f"{avg(metrics['attempts_fail'], fail):.2f}"
        )
        print(
            f"Tiempo promedio request_ticket (red + servidor): "
            f"{avg(metrics['request_time_total'], metrics['request_count']):.6f} s"
        )
        print(
            f"Tiempo promedio purchase (red + servidor): "
            f"{avg(metrics['purchase_time_total'], metrics['purchase_count']):.6f} s"
        )
        print(f"Errores de red detectados: {metrics['network_errors']}")
        print("==============================================\n")


def parse_args():
    parser = argparse.ArgumentParser(description="Cliente de boletos - Práctica 2 Fase 1")
    parser.add_argument("--host", default="127.0.0.1", help="Host del servidor")
    parser.add_argument("--port", type=int, default=5000, help="Puerto del servidor")
    return parser.parse_args()


def wait_for_sales_start(host, port):
    with terminal_lock:
        print("Esperando señal de inicio del servidor...")

    while True:
        try:
            response = send_request(host, port, {"action": "health", "request_id": str(uuid.uuid4())})
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            time.sleep(0.4)
            continue

        if response.get("status") == "ok" and response.get("sales_open"):
            with terminal_lock:
                print("Servidor confirmó inicio de venta.")
            return

        time.sleep(0.3)


def main():
    global sales_start_ts, sales_end_ts

    args = parse_args()

    with terminal_lock:
        print("Cliente de compra iniciado")
        print(f"Servidor destino: {args.host}:{args.port}")
        print(f"Asientos esperados: {TOTAL_ASIENTOS}")
        print(f"Compradores a crear: {TOTAL_COMPRADORES}")

    wait_for_sales_start(args.host, args.port)

    sales_start_ts = time.perf_counter()

    for buyer_id in range(1, TOTAL_COMPRADORES + 1):
        thread = threading.Thread(target=buyer_worker, args=(buyer_id, args.host, args.port), daemon=False)
        threads.append(thread)
        thread.start()
        time.sleep(0.0005)

    for thread in threads:
        thread.join()

    sales_end_ts = time.perf_counter()
    print_summary()


if __name__ == "__main__":
    main()
```